

DIGITAL DESIGN VERIFICATION REPORT

UART Transceiver with TX and RX FIFOs

RTL implementation, self-checking test cases, Vivado XSim waveforms, and results

Protocol	8-N-1 UART, LSB first, internal loopback
Buffering	16x8 synchronous FIFO on both TX and RX sides
Verification	12 simple self-checking test cases in AMD Vivado 2024.1 XSim
Date	11 July 2026

Final result

PASS - all 12 test cases completed with zero errors. XSim reached the final PASS banner at 4098 ns.

1. Executive Summary

This report documents the completed UART loopback design after adding a receiver-side FIFO to the existing transmit-buffered architecture. The design now contains independent 16-byte FIFOs on both sides of the serial link. The TX FIFO absorbs fast producer bursts, while the RX FIFO holds completed bytes until the consumer reads them. The original external receive interface remains simple: `rdy` indicates that at least one byte is buffered, `data_out` shows the oldest byte, and `rdy_clr` pops one byte.

The verification environment deliberately uses basic, readable stimulus. It checks reset/idle behavior, ordinary and edge data patterns, back-to-back TX writes, delayed RX reads, ordered FIFO behavior, reset during activity, post-reset recovery, FIFO full protection, simultaneous FIFO access, and pointer wrap-around. The testbench is self-checking and reports failure with `$fatal` instead of relying only on visual waveform inspection.

Design under test	uart_top with baud generator, TX FSM, RX FSM, TX FIFO, and RX FIFO
Simulation top	uart_top_tb
Simulation parameters	TX_DIV=16, RX_DIV=1 for accelerated 16x oversampling
Test cases	12
Passed / failed	12 / 0
Simulation end	4098 ns
Acceptance result	UART_FIFO_VERIFICATION_PASS

2. Design Architecture

2.1 End-to-end data flow

The internal loopback path is:

```
data_in/wr_en -> TX FIFO -> UART transmitter -> tx_line -> UART receiver
               -> completed rx_byte -> RX FIFO -> data_out/rdy/rdy_clr
```

The baud-rate generator supplies one transmit enable pulse per serial bit and one receiver enable pulse per oversampling position. In the accelerated test configuration, the transmitter advances once every 16 clocks and the receiver advances every clock, preserving the intended 16:1 timing relationship.

2.2 Transmit side

- A write pulse stores `data_in` in the TX FIFO if it is not full.
- When the transmitter is idle and the FIFO is not empty, `tx_start` pops the oldest queued byte and starts a frame.
- The sender emits a low start bit, eight LSB-first data bits, and a high stop bit.

- busy remains high while the sender FSM is outside IDLE.

2.3 Receiver-side FIFO integration

The receiver still reconstructs one byte at a time, but its rdy output is now an internal completion indication called rx_byte_ready. When that indication is high and the RX FIFO has space, uart_top asserts rx_fifo_write. The same pulse acknowledges the receiver, ensuring that each completed byte is enqueued exactly once.

At the external interface, rdy is assigned to the inverse of rx_fifo_empty, data_out is assigned to the FIFO show-ahead output, and rdy_clr becomes the FIFO read enable. A consumer can therefore delay reading several bytes without losing order or overwriting the first unread byte.

RX FIFO full behavior

If the RX FIFO is full, rx_byte_ready remains asserted because the receiver is not acknowledged. Once a location becomes available, the pending byte is written and the receiver ready flag is cleared.

2.4 FIFO implementation

uart_fifo is a 16-entry, 8-bit synchronous circular buffer with show-ahead output. Four-bit read and write pointers select the memory entries, while a five-bit count distinguishes empty from full even when pointer values are equal. Writes while full and reads while empty are ignored. During a simultaneous valid read and write, both pointers advance and the count remains unchanged.

3. Verification Method

3.1 Self-checking structure

1. Drive a simple stimulus using reusable tasks such as send_byte, pop_expect_byte, fifo_write, and fifo_pop_expect.
2. Wait for the exact handshake or FIFO state relevant to the case.
3. Compare the observed value with the expected value using case-aware checks where unknowns matter.
4. Increment error_count and print a FAIL message when any check does not match.
5. Print the final PASS banner only when error_count is zero; otherwise call \$fatal.

A separate timeout process also calls \$fatal if the design stops making progress. This prevents a hung wait statement from being mistaken for success.

3.2 Vivado artifacts

Project	vivado/uart_fifo_verification/uart_fifo_verification.xpr
Wave configuration	vivado/uart_fifo_verification/uart_fifo_waveform.wcfg
Exact waveform data	vivado/uart_fifo_verification/uart_fifo_waveform.vcd
XSim transcript	vivado/.../xsim/vivado_xsim_results.log
Testbench	rtlCode/uart_top_tb.v

4. Test Cases and Results

All cases below were executed in one repeatable XSim run. The end time is the timestamp printed by the testbench after the final assertion for that case passed.

ID	Area	Simple stimulus	Observed result
TC01	Reset / idle	Release reset and inspect idle outputs.	PASS at 10 ns
TC02	Single byte	Send A5 and read it from RX FIFO.	PASS at 350 ns
TC03	All zero	Send 00.	PASS at 670 ns
TC04	All one	Send FF.	PASS at 990 ns
TC05	Alternating	Send 55 and AA.	PASS at 1630 ns
TC06	TX FIFO burst	Queue 11,22,33,44,77,88 without waiting for the UART.	PASS at 3547 ns
TC07	RX FIFO hold/read	Withhold reads, verify count/front, then pop six bytes in order.	PASS at 3590 ns
TC08	Reset during transfer	Queue C3/5A, reset while busy, verify flush and idle.	PASS at 3606 ns
TC09	Post-reset recovery	Send and receive 3C after reset.	PASS at 3946 ns
TC10	FIFO capacity	Fill 16 entries, reject EE overflow, read 0 through 15.	PASS at 4078 ns
TC11	Simultaneous FIFO I/O	Read 5C while writing D2; count remains one.	PASS at 4090 ns
TC12	FIFO wrap-around reuse	Write and read 9B after a complete pointer cycle.	PASS at 4098 ns

4.1 Final XSim transcript

```

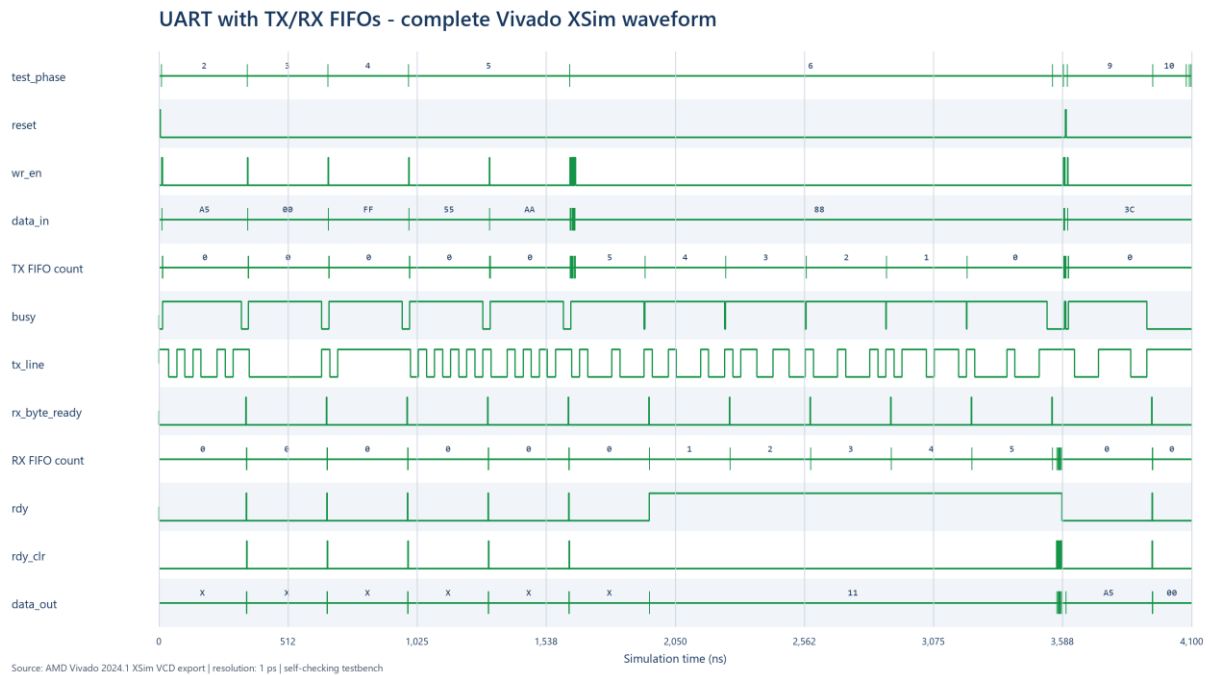
TEST_CASE TC01 PASS reset and idle checks at 10 ns
TEST_CASE TC02 PASS single byte A5 at 350 ns
TEST_CASE TC03 PASS all-zero byte at 670 ns
TEST_CASE TC04 PASS all-one byte at 990 ns
TEST_CASE TC05 PASS alternating patterns at 1630 ns
TEST_CASE TC06 PASS TX FIFO accepted six-byte burst at 3547 ns
TEST_CASE TC07 PASS RX FIFO hold and ordered reads at 3590 ns
TEST_CASE TC08 PASS reset flush during transfer at 3606 ns
TEST_CASE TC09 PASS post-reset recovery at 3946 ns
TEST_CASE TC10 PASS FIFO full and overflow protection at 4078 ns
TEST_CASE TC11 PASS simultaneous FIFO read and write at 4090 ns
TEST_CASE TC12 PASS FIFO wrap-around reuse at 4098 ns
UART_FIFO_VERIFICATION_PASS: all 12 test cases passed at 4098 ns

```

5. Exact Waveform Evidence

The following figures are rendered from the VCD exported by AMD Vivado 2024.1 XSim. The VCD time resolution is 1 ps, so the transitions and bus values come from the simulator database rather than from a hand-drawn timing diagram.

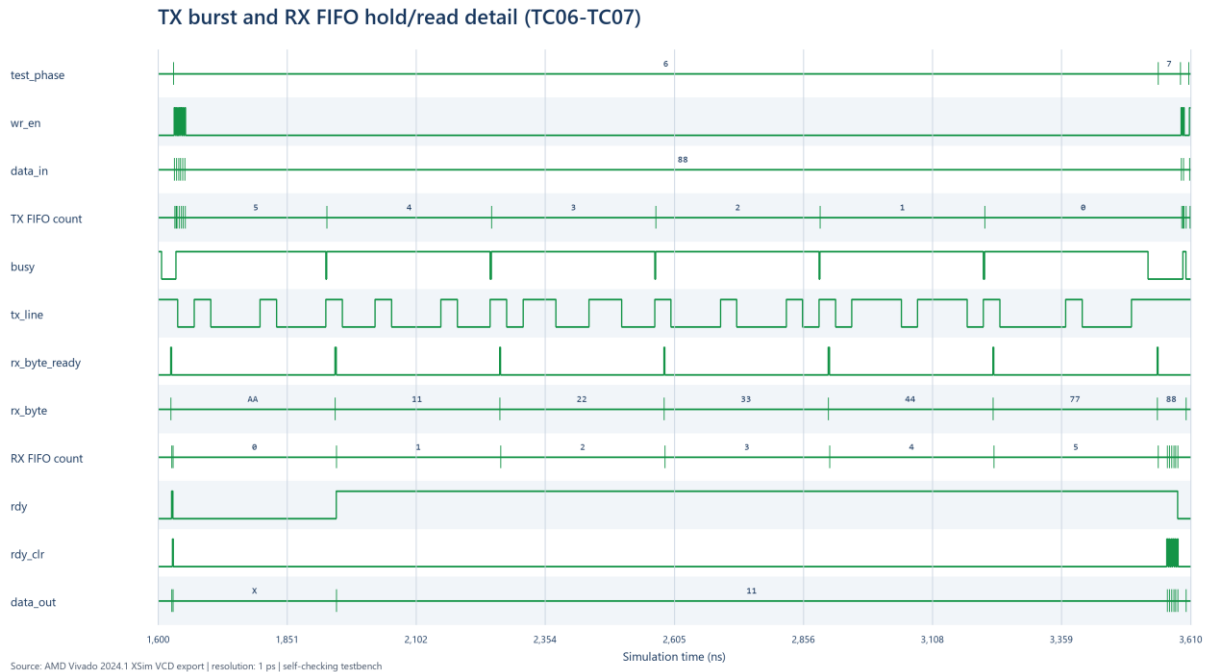
Figure 1. Complete 0-4100 ns UART and FIFO verification waveform



5.1 Overview explanation

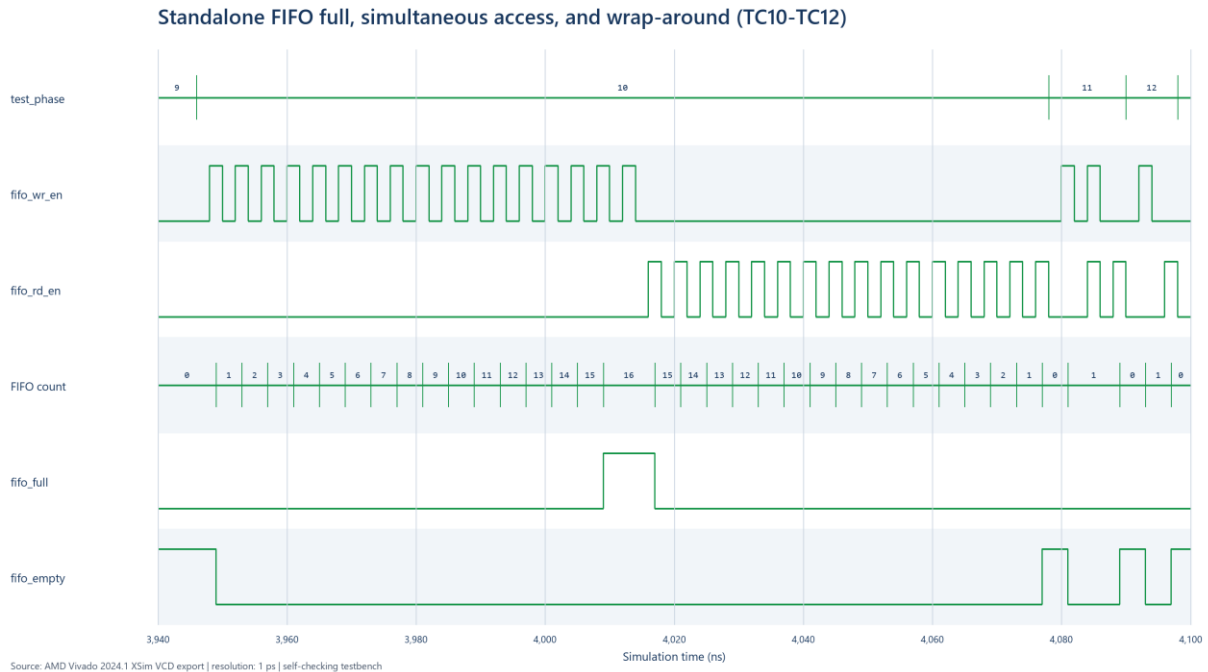
- Phases 2-5 show A5, 00, FF, 55, and AA passing through the complete UART path.
- Each `wr_en` pulse places a byte into the TX path; `busy` stays high during its serial frame and `tx_line` shows the 8-N-1 bit sequence.
- `rx_byte_ready` pulses once after each valid stop bit. For the single-byte cases, `rdy_clr` immediately pops the received byte.
- During phase 6, TX FIFO count falls from five to zero as the six queued bytes are serialized, while RX FIFO count rises because reads are intentionally withheld.
- The reset pulse near 3590 ns clears both FIFOs, lowers `busy`/`rdy`, and returns `tx_line` to its high idle state.

Figure 2. TX burst and RX FIFO accumulation/dequeue detail (TC06-TC07)



5.2 TX and RX FIFO explanation

- Six narrow `wr_en` pulses enqueue 11, 22, 33, 44, 77, and 88 much faster than the UART can transmit them.
- TX FIFO count begins at five because the first byte is transferred to the sender while the remaining five wait in the queue.
- Each completed frame creates one `rx_byte_ready` pulse and advances `rx_byte` through 11, 22, 33, 44, 77, and 88.
- RX FIFO count climbs from zero to six while no consumer reads occur. `rdy` remains high and `data_out` remains 11, proving show-ahead behavior and stable oldest-byte ordering.
- At the end of phase 7, six `rdy_clr` pulses rapidly consume the queue; count returns to zero and `rdy` falls.

Figure 3. Standalone FIFO capacity, simultaneous access, and wrap-around detail (TC10-TC12)

5.3 FIFO unit behavior

- Sixteen write pulses increment count from zero to 16. `fifo_full` asserts exactly at the configured depth.
- The extra overflow write does not increase count beyond 16.
- Sixteen read pulses return count to zero and assert `fifo_empty`.
- In TC11, read and write are high together; occupancy remains one and D2 becomes the new front value.
- TC12 performs another write/read after the pointers have wrapped, demonstrating continued correct reuse.

6. Conclusion

The receiver-side FIFO is fully integrated without changing the top-level receive handshake. The design now buffers bursts on both sides of the UART, preserves byte order, handles delayed reads, flushes cleanly on reset, and recovers for later communication. Vivado XSim verified every planned basic case, and the saved WCFG/VCD artifacts make the result repeatable and inspectable.

Result

12/12 self-checking cases passed, zero reported errors, final simulation time 4098 ns.

6.1 Sensible next improvements

- Expose TX-full, RX-full, or available-count status to external logic.
- Add a two-flop synchronizer for a real asynchronous RX pin.
- Report framing error and optional parity error conditions.
- Use majority voting around the center sample for improved noise tolerance.
- Make data bits, parity, stop bits, FIFO depth, clock frequency, and baud rate configurable.

Appendix A - Integrated UART Top with TX and RX FIFOs

Simulated source: rtlCode/uart_top.v

```
`timescale 1ns/1ps

module uart_top #(
    parameter integer TX_DIV = 5208,
    parameter integer RX_DIV = 325
) (
    input clk,
    input reset,
    input [7:0] data_in,
    input wr_en,
    input rdy_clr,
    output rdy,
    output busy,
    output [7:0] data_out
);
    wire tx_clock_enable;
    wire rx_en;
    wire tx_line;

    // TX FIFO (buffers user writes so they can be faster than UART line rate)
    wire tx_fifo_full;
    wire tx_fifo_empty;
    wire [7:0] tx_fifo_dout;

    // Transmitter busy (state != IDLE)
    wire tx_busy;

    // When transmitter is idle and FIFO has data, pop 1 byte and start TX.
    wire tx_start = (!tx_fifo_empty) && (tx_busy == 1'b0);

    baud_rate_generator #(
        .TX_DIV(TX_DIV),
        .RX_DIV(RX_DIV)
    ) brg (
        .clk(clk),
        .reset(reset),
        .tx_enable(tx_clock_enable),
        .rx_en(rx_en)
    );

    uart_fifo tx_fifo (
        .clk(clk),
        .reset(reset),
        .wr_en(wr_en),

        .wr_data(data_in),
        .full(tx_fifo_full),
        .rd_en(tx_start),
        .rd_data(tx_fifo_dout),
        .empty(tx_fifo_empty)
    );

    uart_sender tx_inst (
        .clk(clk),
        .rst(reset),
        .wr_en(tx_start),
        .tx_en(tx_clock_enable),
        .data_in(tx_fifo_dout),
        .tx(tx_line),
        .busy(tx_busy)
    );
endmodule
```

```

// Keep original meaning: `busy` = transmitter busy.
assign busy = tx_busy;

// RX FIFO (stores complete received bytes until the user reads them).
wire rx_byte_ready;
wire [7:0] rx_byte;
wire rx_fifo_full;
wire rx_fifo_empty;
wire [7:0] rx_fifo_dout;

// Capture each receiver result once. If the FIFO is full, the receiver
// keeps rx_byte_ready asserted until a FIFO location becomes available.
wire rx_fifo_write = rx_byte_ready && !rx_fifo_full;
wire rx_fifo_read = rdy_clr && !rx_fifo_empty;

uart_receiver rx_inst (
    .clk(clk),
    .rst(reset),
    .rx(tx_line),
    .rdy_clr(rx_fifo_write),
    .rx_en(rx_en),
    .rdy(rx_byte_ready),
    .data_out(rx_byte)
);

uart_fifo rx_fifo (
    .clk(clk),
    .reset(reset),
    .wr_en(rx_fifo_write),
    .wr_data(rx_byte),
    .full(rx_fifo_full),
    .rd_en(rx_fifo_read),
    .rd_data(rx_fifo_dout),
    .empty(rx_fifo_empty)
);

// Preserve the original user interface: rdy means at least one byte is
// buffered, data_out shows the oldest byte, and rdy_clr consumes it.
assign rdy = !rx_fifo_empty;
assign data_out = rx_fifo_dout;
endmodule

```

Appendix B - 16x8 Synchronous FIFO

Simulated source: rtlCode/uart_fifo.v

```

`timescale 1ns/1ps

// Minimal 16-deep, 8-bit synchronous FIFO (single clock).
// - Write when `wr_en` and not `full`
// - Read (pop) when `rd_en` and not `empty`
// - `rd_data` is "show-ahead" (always shows the front element)
module uart_fifo (
    input clk,
    input reset,

    input wr_en,
    input [7:0] wr_data,
    output full,

    input rd_en,
    output [7:0] rd_data,
    output empty
);
    reg [7:0] mem [0:15];
    reg [3:0] wr_ptr;
    reg [3:0] rd_ptr;
    reg [4:0] count;

    assign empty = (count == 5'd0);
    assign full = (count == 5'd16);

    assign rd_data = mem[rd_ptr];

    wire do_write = wr_en && !full;
    wire do_read = rd_en && !empty;

    always @(posedge clk) begin
        if (reset) begin

```

```

        wr_ptr <= 4'd0;
        rd_ptr <= 4'd0;
        count <= 5'd0;
    end else begin
        if (do_write) begin
            mem[wr_ptr] <= wr_data;
            wr_ptr <= wr_ptr + 4'd1;
        end

        if (do_read) begin
            rd_ptr <= rd_ptr + 4'd1;
        end

        case ({do_write, do_read})
            2'b10: count <= count + 5'd1;
            2'b01: count <= count - 5'd1;
            default: count <= count;
        endcase
    end
end
endmodule

```

Appendix C - Self-checking Vivado Testbench

Simulated source: rtlCode/uart_top_tb.v

```

`timescale 1ns/1ps

module uart_top_tb;
    reg clk;
    reg reset;
    reg [7:0] data_in;
    reg wr_en;
    reg rdy_clr;

    // Dedicated FIFO stimulus for simple unit-level capacity tests.
    reg fifo_reset;
    reg fifo_wr_en;
    reg [7:0] fifo_wr_data;
    reg fifo_rd_en;
    wire fifo_full;
    wire fifo_empty;
    wire [7:0] fifo_rd_data;

    wire rdy;
    wire busy;
    wire [7:0] data_out;

    integer error_count;
    integer test_phase;
    integer i;

    reg [7:0] burst_data [0:5];

    uart_top #(
        .TX_DIV(16),
        .RX_DIV(1)
    ) dut (
        .clk(clk),
        .reset(reset),
        .data_in(data_in),
        .wr_en(wr_en),
        .rdy_clr(rdy_clr),
        .rdy(rdy),
        .busy(busy),
        .data_out(data_out)
    );

    uart_fifo fifo_unit (
        .clk(clk),
        .reset(fifo_reset),
        .wr_en(fifo_wr_en),
        .wr_data(fifo_wr_data),
        .full(fifo_full),
        .rd_en(fifo_rd_en),
        .rd_data(fifo_rd_data),
        .empty(fifo_empty)
    );

    // Fast simulation clock so complete UART frames appear quickly in the waveform.

```

```

always #1 clk = ~clk;

task check_value;
  input condition;
  input [8*80-1:0] message;
  begin
    if (!condition) begin
      error_count = error_count + 1;
      $display("FAIL: %0s at %0t", message, $time);
    end
  end
endtask

task pulse_reset;
  begin
    @(negedge clk);
    reset = 1'b1;
    @(negedge clk);
    reset = 1'b0;
    repeat (2) @(negedge clk);
  end
endtask

task send_byte;
  input [7:0] d;
  begin
    @(negedge clk);
    data_in = d;
    wr_en = 1'b1;
    @(negedge clk);
    wr_en = 1'b0;
  end
endtask

task clear_ready;
  begin
    @(negedge clk);
    rdy_clr = 1'b1;
    @(negedge clk);
    rdy_clr = 1'b0;
  end
endtask

task pop_expect_byte;
  input [7:0] exp;
  begin
    wait (rdy == 1'b1);
    if (data_out != exp) begin
      error_count = error_count + 1;
      $display("FAIL: expected RX byte %02h, got %02h at %0t", exp, data_out, $time);
    end
    clear_ready();
  end
endtask

task fifo_write;
  input [7:0] d;
  begin
    @(negedge clk);
    fifo_wr_data = d;
    fifo_wr_en = 1'b1;
    @(negedge clk);
    fifo_wr_en = 1'b0;
  end
endtask

task fifo_pop_expect;
  input [7:0] exp;
  begin
    @(negedge clk);
    if (fifo_rd_data != exp) begin
      error_count = error_count + 1;
      $display("FAIL: expected FIFO byte %02h, got %02h at %0t", exp, fifo_rd_data, $time);
    end
    fifo_rd_en = 1'b1;
    @(negedge clk);
    fifo_rd_en = 1'b0;
  end
endtask

initial begin
  $timeformat(-9, 0, " ns", 8);
  clk = 1'b0;
  reset = 1'b1;
  data_in = 8'd0;

```

```

wr_en = 1'b0;
rdy_clr = 1'b0;

fifo_reset = 1'b1;
fifo_wr_en = 1'b0;
fifo_wr_data = 8'd0;
fifo_rd_en = 1'b0;

error_count = 0;
test_phase = 0;

burst_data[0] = 8'h11;
burst_data[1] = 8'h22;
burst_data[2] = 8'h33;
burst_data[3] = 8'h44;
burst_data[4] = 8'h77;
burst_data[5] = 8'h88;

repeat (3) @(negedge clk);
reset = 1'b0;
fifo_reset = 1'b0;
repeat (2) @(negedge clk);

// TC01: reset and idle outputs.
test_phase = 1;
check_value(busy === 1'b0, "TC01 transmitter idle after reset");
check_value(rdy === 1'b0, "TC01 RX FIFO empty after reset");
check_value(dut.tx_line === 1'b1, "TC01 serial line idle high");
check_value(dut.tx_fifo_empty === 1'b1, "TC01 TX FIFO empty after reset");
check_value(dut.rx_fifo_empty === 1'b1, "TC01 RX FIFO empty flag after reset");
$display("TEST_CASE TC01 PASS reset and idle checks at %0t", $time);

// TC02-TC05: basic data patterns exercise ordinary and edge bit values.
test_phase = 2;
send_byte(8'hA5);
pop_expect_byte(8'hA5);
$display("TEST_CASE TC02 PASS single byte A5 at %0t", $time);

test_phase = 3;
send_byte(8'h00);
pop_expect_byte(8'h00);
$display("TEST_CASE TC03 PASS all-zero byte at %0t", $time);

test_phase = 4;
send_byte(8'hFF);
pop_expect_byte(8'hFF);
$display("TEST_CASE TC04 PASS all-one byte at %0t", $time);

test_phase = 5;
send_byte(8'h55);
pop_expect_byte(8'h55);
send_byte(8'hAA);
pop_expect_byte(8'hAA);
$display("TEST_CASE TC05 PASS alternating patterns at %0t", $time);

// TC06: queue a burst while TX is busy; the TX FIFO must preserve order.
test_phase = 6;
for (i = 0; i < 6; i = i + 1) begin
    send_byte(burst_data[i]);
end
wait (dut.rx_fifo.count == 5'd6);
check_value(dut.tx_fifo_empty === 1'b1, "TC06 TX FIFO drains after burst");
$display("TEST_CASE TC06 PASS TX FIFO accepted six-byte burst at %0t", $time);

// TC07: do not read during reception; RX FIFO must hold all six bytes.
test_phase = 7;
check_value(rdy === 1'b1, "TC07 ready stays high while RX FIFO has data");
check_value(data_out === 8'h11, "TC07 RX FIFO front is first burst byte");
repeat (8) @(negedge clk);
check_value(data_out === 8'h11, "TC07 front byte remains stable until pop");
for (i = 0; i < 6; i = i + 1) begin
    pop_expect_byte(burst_data[i]);
end
repeat (2) @(negedge clk);
check_value(rdy === 1'b0, "TC07 ready clears after last RX FIFO pop");
check_value(dut.rx_fifo_empty === 1'b1, "TC07 RX FIFO empty after ordered reads");
$display("TEST_CASE TC07 PASS RX FIFO hold and ordered reads at %0t", $time);

// TC08: reset during an active transfer flushes both FIFOs and FSM state.
test_phase = 8;
send_byte(8'hC3);

send_byte(8'h5A);
wait (busy === 1'b1);
pulse_reset();
check_value(busy === 1'b0, "TC08 transmitter stops on reset");

```

```

check_value(rdy === 1'b0, "TC08 receiver queue cleared on reset");
check_value(dut.tx_fifo_empty === 1'b1, "TC08 TX FIFO cleared on reset");
check_value(dut.rx_fifo_empty === 1'b1, "TC08 RX FIFO cleared on reset");
check_value(dut.tx_line === 1'b1, "TC08 line returns to idle high");
$display("TEST_CASE TC08 PASS reset flush during transfer at %0t", $time);

// TC09: confirm normal communication resumes after reset.
test_phase = 9;
send_byte(8'h3C);
pop_expect_byte(8'h3C);
$display("TEST_CASE TC09 PASS post-reset recovery at %0t", $time);

// TC10: fill a standalone FIFO to 16 entries and verify overflow is ignored.
test_phase = 10;
for (i = 0; i < 16; i = i + 1) begin
    fifo_write(i[7:0]);
end
check_value(fifo_full === 1'b1, "TC10 FIFO full at depth 16");
fifo_write(8'hEE);
check_value(fifo_unit.count === 5'd16, "TC10 write while full is ignored");
for (i = 0; i < 16; i = i + 1) begin
    fifo_pop_expect(i[7:0]);
end
check_value(fifo_empty === 1'b1, "TC10 FIFO empty after 16 reads");
$display("TEST_CASE TC10 PASS FIFO full and overflow protection at %0t", $time);

// TC11: simultaneous read/write keeps occupancy constant and replaces data.
test_phase = 11;
fifo_write(8'h5C);
@(negedge clk);
check_value(fifo_rd_data === 8'h5C, "TC11 initial FIFO front value");
fifo_wr_data = 8'hD2;
fifo_wr_en = 1'b1;
fifo_rd_en = 1'b1;
@(negedge clk);
fifo_wr_en = 1'b0;
fifo_rd_en = 1'b0;
check_value(fifo_unit.count === 5'd1, "TC11 simultaneous read/write keeps count");
check_value(fifo_rd_data === 8'hD2, "TC11 new byte becomes FIFO front");
fifo_pop_expect(8'hD2);

$display("TEST_CASE TC11 PASS simultaneous FIFO read and write at %0t", $time);

// TC12: prior full-cycle pointer wrap did not damage reuse.
test_phase = 12;
fifo_write(8'h9B);
fifo_pop_expect(8'h9B);
check_value(fifo_empty === 1'b1, "TC12 FIFO reusable after pointer wrap");
$display("TEST_CASE TC12 PASS FIFO wrap-around reuse at %0t", $time);

test_phase = 13;
if (error_count == 0) begin
    $display("UART_FIFO_VERIFICATION_PASS: all 12 test cases passed at %0t", $time);
    $finish;
end else begin
    $display("UART_FIFO_VERIFICATION_FAIL: %0d errors at %0t", error_count, $time);
    $fatal(1, "UART/FIFO verification failed");
end
end

initial begin
    #20000;
    $fatal(1, "Simulation timeout");
end
endmodule

```